# FUNDAMENTOS DE PROGRAMACIÓN. Curso 2022/23

**PRIMER EXAMEN PARCIAL. 23 de enero de 2023. Soluciones**

## Ejercicio 1

```
def parsea_y_suma_resultados(resultados_str):
    puntos_eq1, puntos_eq2 = 0, 0
    for parcial in resultados_str.split("*"):
        eq1, eq2 = parcial.split("-")
        puntos_eq1 += int(eq1)
        puntos_eq2 += int(eq2)
    return puntos_eq1, puntos_eq2
```

## Ejercicio 2

```
def lee_datos_baloncesto(fichero):
    res = []
    with open(fichero, encoding="utf-8") as f:
        lector = csv.reader(f, delimiter=";")
        next(lector)
        for fecha, eq1, eq2, competicion, resultados, faltas1, faltas2 in lector:
            fecha = datetime.strptime(fecha, "%d/%m/%Y").date()
            puntos1, puntos2 = parsea_y_suma_resultados(resultados)
            faltas1, faltas2 = int(faltas1), int(faltas2)
            tupla = PartidoBasket(fecha, eq1, eq2, competicion,
                puntos1, puntos2, faltas1, faltas2)
            res.append(tupla)
    return res
```

## Ejercicio 3

```
def equipo_mas_faltas(lista_partidos, equipos=None):
    dicc = defaultdict(int)
    for p in lista_partidos:
        if equipos is None or p.equipo1 in equipos:
            dicc[p.equipo1] += p.faltas_eq1
        if equipos is None or p.equipo2 in equipos:
            dicc[p.equipo2] += p.faltas_eq2
    return max(dicc.items(), key=lambda x: x[1])
    # O devolver solo el nombre:
    # return max(dicc.items(), key=lambda x: x[1])[0]
```

## Ejercicio 4

```
# Con un diccionario de agrupación, y uno de medias
def media_puntos_por_equipo(lista_partidos, competicion):
    dicc_puntos = defaultdict(list)
    for p in lista_partidos:
        if p.competicion == competicion:
```

```
            dicc_puntos[p.equipo1].append(p.puntos_eq1)
            dicc_puntos[p.equipo2].append(p.puntos_eq2)
    res = {}
    for equipo, puntos in dicc_puntos.items():
        res[equipo] = sum(puntos) / len(puntos)
    return res

# Solución alternativa
def media_puntos_por_equipo(lista_partidos, competicion):
    dicc_puntos = defaultdict(int)
    contador_equipos = defaultdict(int)
    for p in lista_partidos:
        if p.competicion == competicion:
            dicc_puntos[p.equipo1] += p.puntos_eq1
            dicc_puntos[p.equipo2] += p.puntos_eq2
            contador_equipos[p.equipo1] += 1
            contador_equipos[p.equipo2] += 1

    res = {}
    for equipo, puntos in dicc_puntos.items():
        res[equipo] = puntos / contador_equipos[equipo]
    return res
```

**Ejercicio 5**

```
def diferencia_puntos_anotados(lista_partidos, equipo):
   resultados_equipo = []
   for p in lista_partidos:
        if p.equipo1 == equipo:
              resultados_equipo.append((p.fecha, p.puntos_eq1))
        elif p.equipo2 == equipo:
              resultados_equipo.append((p.fecha, p.puntos_eq2))
    resultados_equipo = sorted(resultados_equipo)
    return [p2[1] - p1[1] for p1, p2 in zip(resultados_equipo, resultados_equipo[1:])]

# Solución alternativa
def diferencia_puntos_anotados(lista_partidos, equipo):
    resultados_eq1 = [
        (p.fecha, p.puntos_eq1) for p in lista_partidos if p.equipo1 == equipo
    ]
    resultados_eq2 = [
        (p.fecha, p.puntos_eq2) for p in lista_partidos if p.equipo2 == equipo
    ]
    resultados_equipo = sorted(resultados_eq1 + resultados_eq2)
    return [p2[1] - p1[1] for p1, p2 in zip(resultados_equipo, resultados_equipo[1:])]
    # Usando índices:
    # return [resultados_equipos[i] - resultados_equipos[i-1]
                                          for i in range(1,len(resultados_esquipos)])
```

### Ejercicio 6

```
def victorias_por_equipo(lista_partidos):
    dicc_cont = defaultdict(int)
    for p in lista_partidos:
       if p.puntos_eq1 > p.puntos_eq2:
              dicc_cont[p.equipo1] += 1
       elif p.puntos_eq2 > p.puntos_eq1:
              dicc_cont[p.equipo2] += 1
    return dicc_cont

# Solución alternativa
def victorias_por_equipo(lista_partidos):
    dicc_cont = Counter(p.equipo1 for p in lista_partidos if p.puntos_eq1 > p.puntos_eq2)
              + Counter(p.equipo2 for p in lista_partidos if p.puntos_eq2 > p.puntos_eq1)
    return dicc_cont
```

### Ejercicio 7

```
def equipos_minimo_victorias(lista_partidos, n):
    dicc = victorias_por_equipo(lista_partidos)
    return sorted((equipo for equipo, victorias in dicc.items() if victorias >= n),
        key=lambda t:t[1])
    # Para hacerlo con most_common, debe haberse resuelto el anterior con Counter:
    # return [equipo for equipo, victorias in dicc.most_common() if victorias >= n]
```

### Ejercicio 8

```
def equipos_mas_victorias_por_año(lista_partidos, n):
    partidos_por_año = defaultdict(list)
    for p in lista_partidos:
        partidos_por_año[p.fecha.year].append(p)
    res = {}
    for año, partidos_año in partidos_por_año.items():
        res[año] = equipos_minimo_victorias(partidos_año, n)
    return res
```

### Ejercicio 9

```
def test_lee_datos_baloncesto(datos):
    print("Test de lee_datos_baloncesto:")
    print("Total registros leídos:", len(datos))
    print("Mostrando los tres primeros registros:")
    for p in datos[:3]:
        print("\t", p)
    print()
```

```python
def test_busca_equipo_mas_faltas(datos):
    print("Test de busca_equipo_mas_faltas:")
    print(busca_equipo_mas_faltas(datos))
    print()

def test_calcula_media_puntos_por_equipo(datos, competicion):
    print(f"Test de calcula_media_puntos_por_equipo (competicion={competicion}):")
    datos = lee_datos_baloncesto("data/resultados_baloncesto.csv")
    print(calcula_media_puntos_por_equipo(datos, competicion))
    print()

def tests_calcula_diferencia_puntos_anotados(datos, equipo):
    print(f"Test de calcula_diferencia_puntos_anotados (equipo={equipo})")
    print(calcula_diferencia_puntos_anotados(datos, equipo))
    print()

def test_cuenta_victorias_por_equipo(datos):
    print("Test de cuenta_victorias_por_equipo")
    print(cuenta_victorias_por_equipo(datos))
    print()

def test_filtra_equipos_minimo_victorias(datos, n):
    print(f"Test de filtra_equipos_minimo_victorias (n={n})")
    print(filtra_equipos_minimo_victorias(datos, n))
    print()

def test_calcula_equipos_mas_victorias_por_año(datos, n):
    print(f"Test de calcula_equipos_mas_victorias_por_año (n={n})")
    datos = lee_datos_baloncesto("data/resultados_baloncesto.csv")
    print(calcula_equipos_mas_victorias_por_año(datos, n))
    print()

if __name__=="__main__":
    datos = lee_datos_baloncesto("data/resultados_baloncesto.csv")
    test_lee_datos_baloncesto(datos)
    test_busca_equipo_mas_faltas(datos)
    test_calcula_media_puntos_por_equipo(datos, "Copa del Rey")
    tests_calcula_diferencia_puntos_anotados(datos, "Barcelona")
    test_cuenta_victorias_por_equipo(datos)
    test_filtra_equipos_minimo_victorias(datos, 8)
    test_calcula_equipos_mas_victorias_por_año(datos, 8)
```